

AI BUILD CHALLENGE

Build a Small Real-World Project Using AI, Git & GitHub

Goal: Learn how to turn an idea into a working application with the help of an LLM, test it, improve it, and publish the source code on GitHub.

Important: AI can help you write the code, but you should understand what your project does, test it yourself, and be able to explain the main parts.

Common Rules for All Years

1. Choose one project from your year's list.
2. Use ChatGPT, Gemini, Claude, or another LLM to help build the project.
3. Start with a clear prompt. If something does not work, ask the AI to debug or explain it.
4. Add at least **one improvement** after the first version works.
5. Test the application yourself.
6. Use Git to track the project and upload it to a **public GitHub repository**.
7. Submit the GitHub repository link in the provided form.

1st YEAR TASK

Choose **ONE** of the following beginner-friendly projects. These projects focus on basic HTML, CSS and JavaScript concepts.

1. ID Generator

What to build: Create a simple form that takes details such as name, USN/ID, branch or department and generates a clean digital ID. Include validation, a generated ID number, and a reset button. Optional: add photo upload and print/download.

Example AI prompt:

Create a simple ID Generator using HTML, CSS and JavaScript. Add fields for name, student ID, branch and college. When the user submits the form, display a professional ID card with the entered details and a generated unique ID. Add validation and a reset button. Keep it beginner-friendly.

Suggested improvement: Add one extra feature such as better validation, dark mode, print/download, search, or a better UI.

2. Password Generator

What to build: Build a tool that generates random passwords. Let the user choose password length and whether to include uppercase letters, lowercase letters, numbers and symbols. Add a copy button and a simple strength indicator.

Example AI prompt:

Create a beginner-friendly Password Generator using HTML, CSS and JavaScript. Let the user choose password length and character types such as uppercase, lowercase, numbers and symbols. Generate a random password, show its strength, and add a Copy button. Explain the JavaScript.

Suggested improvement: Add one extra feature such as better validation, dark mode, print/download, search, or a better UI.

3. Student Leaderboard

What to build: Enter student names and scores, then display them in ranking order. Show rank, name and score. Add the ability to add students, sort scores, and reset the leaderboard.

Example AI prompt:

Create a Student Leaderboard using HTML, CSS and JavaScript. Allow users to enter student names and marks/scores. Store the entries in an array, sort students from highest to lowest score, assign ranks, display the leaderboard, and add buttons to reset or remove entries. Keep the code simple.

Suggested improvement: Add one extra feature such as better validation, dark mode, print/download, search, or a better UI.

4. Random Team Generator

What to build: Enter a list of student names and choose the number of teams. Randomly divide the students into teams and display the result. Add a regenerate button and handle cases where the number of students is not evenly divisible.

Example AI prompt:

Create a Random Team Generator using HTML, CSS and JavaScript. Let the user enter multiple student names and choose the number of teams. Randomly distribute the students into teams, display each team clearly, and add a Regenerate button. Handle uneven team sizes and explain the logic.

Suggested improvement: Add one extra feature such as better validation, dark mode, print/download, search, or a better UI.

2nd & 3rd YEAR TASK

Choose **ONE**. These projects introduce more application logic, data handling and, where appropriate, database concepts. Students may use a simple frontend with a backend/database if they are comfortable.

1. ATM Simulator

What to build: Create a simulated ATM application. The user enters a PIN, then can check balance, deposit money, withdraw money and view a simple transaction history. Include validation such as preventing withdrawals above the available balance. For a stronger version, store users and transactions in a database.

Example AI prompt:

Build an ATM Simulator using a simple frontend and beginner-friendly JavaScript. Include a PIN screen, balance, deposit, withdrawal, transaction history and logout. Prevent invalid withdrawals and show useful error messages. Structure the code clearly so it can later be connected to a database.

Suggested improvement: Add authentication, database storage, search/filtering, analytics, better validation, or another meaningful feature appropriate to the project.

2. Personal Expense Tracker

What to build: Build an application where a user records expenses with amount, category, date and description. Show total spending, category-wise spending, recent transactions and filters. A stronger version can save the data in a database and provide a simple dashboard.

Example AI prompt:

Create a Personal Expense Tracker using HTML, CSS and JavaScript. Allow the user to add, edit and delete expenses with amount, category, date and description. Calculate total expenses, show recent transactions and allow filtering by category. Use localStorage for the first version and explain how the data could later be stored in a database.

Suggested improvement: Add authentication, database storage, search/filtering, analytics, better validation, or another meaningful feature appropriate to the project.

3. Campus Lost & Found

What to build: Create a small portal for students to report lost or found items. Each record can contain item name, category, description, location, date and contact information. Users should be able to search and filter items and mark an item as recovered. A database-backed version should store users and item records.

Example AI prompt:

Build a Campus Lost and Found web application. Create forms for posting lost or found items with item name, category, description, location, date and contact details. Display the records in cards, add search and category filters, and allow an item to be marked as recovered. Keep the architecture simple and explain how the records can be stored in a database.

Suggested improvement: Add authentication, database storage, search/filtering, analytics, better validation, or another meaningful feature appropriate to the project.

4. Food Chatbot

What to build: Create a simple college/canteen food chatbot. The user can ask questions such as what food is available, prices, vegetarian options, opening hours and recommendations. The first version can use a fixed menu stored in JavaScript/JSON; an optional stronger version can connect to an AI API or database.

Example AI prompt:

Create a simple Food Chatbot for a college canteen using HTML, CSS and JavaScript. Store a small menu in JSON and make the chatbot answer questions about food names, prices, categories, vegetarian options and opening hours. Add a simple chat interface and fallback message for unknown questions. Keep it beginner-friendly.

Suggested improvement: Add authentication, database storage, search/filtering, analytics, better validation, or another meaningful feature appropriate to the project.

Database Guidance for 2nd & 3rd Year

If you choose to use a database, you can use a beginner-friendly option such as MySQL, SQLite, MongoDB or Supabase. Do not make the database unnecessarily complicated.

Example for Personal Expense Tracker:

Table / Collection	Example fields
users	id, name, email
expenses	id, user_id, amount, category, date, description
categories	id, name

Students do not need advanced database design. They should understand the basic idea of a table/collection, record/document, primary identifier, and how the application saves and retrieves data.

Git & GitHub Submission

Git tracks changes in your project on your computer. **GitHub** stores the repository online so it can be shared and evaluated.

Command	Simple meaning
git init	Start Git tracking in the project folder.
git status	See changed/untracked files.
git add .	Stage the project files.
git commit -m "Initial project"	Save a version of the project.
git log	View previous commits.

Basic workflow: Create project → use AI → run and test → fix errors → add an improvement → git init → git add . → git commit → create a public GitHub repository → connect remote → git push → submit the GitHub link.

Important: Do not upload passwords, API keys, private tokens or other secrets to GitHub.

What to Submit

- A working project.
- Source code uploaded to a **public GitHub repository**.
- At least one Git commit.
- Evidence of using an LLM during development (for example, keep your prompts/screenshots or briefly mention how AI helped).
- At least one meaningful improvement after the first working version.
- GitHub repository link submitted through the provided form.

Suggested Evaluation

Functionality — 40% | Code understanding — 20% | UI/UX — 15% | AI usage & improvement — 10% | Git/GitHub — 15%